Adelaide University

Microcontroller Programming

Project Resources

Find object, Drive/Rotate certain Distance/Angle and overall FSM

NUCLEO-G474RE • FSM •Ultrasonic sensor •Drive/Rotate subroutines • Kobuki bytestream

Student Edition

Practical outcome: N/A.

Contents

1 Find and move towards object **1**

2 Drive Distance and Rotate Angle - P controller **2**

3 Overall Project FSM **12**

1. Find and move towards object

Figure 1 highlights how the Kobuki may detect an object and move towards it, yet if it is far away, the robot might not move directly towards it and possibly also never bump into it.

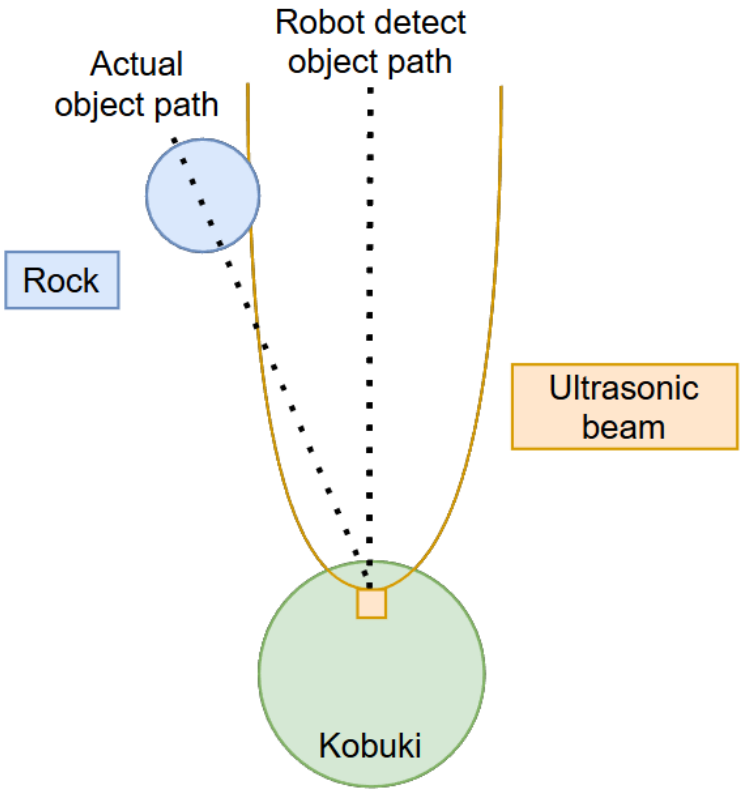
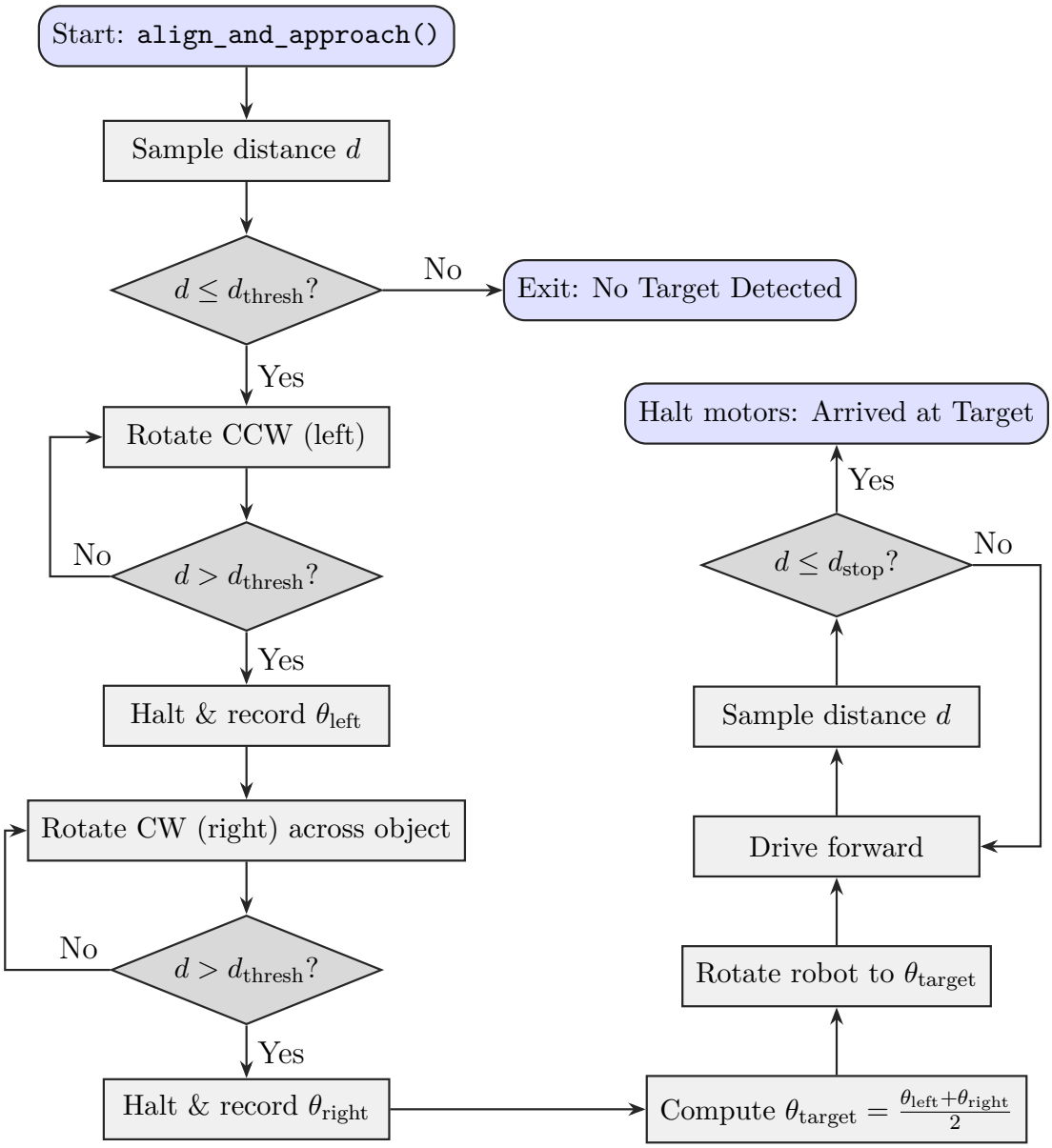


Figure 1: Example of ultrasonic beam pattern detecting an object.

An ultrasonic sensor detects an obstacle by transmitting high-frequency acoustic waves and measuring the total round-trip time it takes for those sound pulses to reflect back from the target surface. However, because sound spreads outwards in a three-dimensional cone rather than travelling along a line, the sensor cannot determine the exact bearing of an object within its detection field. As shown in the Figure 1, a rock positioned far out on the peripheral edge of the ultrasonic beam will still trigger a valid distance return. If the Kobuki robot simply treats every detection as though the target lies along its central forward axis, it will steer along the wrong direction and end up driving past the obstacle entirely rather than centreing on it or making contact. For this reason, it is very important for your group to understand the characteristics of your sensor, and the limitations to detecting an object. From here, you can set up a more complex subroutine to align your Kobuki right at the centre of the object. This is highly related to your first step and final step of the project where you need to orientate your Kobuki correctly.

To address this limitation, you can implement a scanning subroutine in C that measures the full angular span of the obstacle before committing to a forward drive. By setting the Kobuki to rotate in place, your code can detect the exact headings where the object first enters the sensor's cone of vision and where it finally drops out of range. Averaging these

two boundary angles yields the true angular centreline of the target. The subroutine can then turn the robot directly onto this computed heading and advance forward smoothly, ensuring the Kobuki navigates directly to the centre of the object instead of drifting off course. The following flowchart could be beneficial for you:



You may observe that in order to execute this subroutine, you Kobuki is required to rotate and drive at specific angles and distances. We will explore how to do this in the next section.

2. Drive Distance and Rotate Angle - P controller

The following section is credited to Dr. Matthew Forbes.

So far, you can see that we commanded the robot to drive and rotate based on our buttons and sensors' readings or for a certain delay time. For the project, we want to make the

robot move more reliably than that. That is, we need to be able to program the robot to drive for a certain distance and rotate for a certain angle. In order to do so, we first set an exact target destination, then continuously check the robot's progress and adjust its speed until it arrives. This process is handled by two separate functions each for driving and rotating.

The `Kobuki_Drive_Distance_Setpoint` and `Kobuki_Rotate_Distance_Setpoint` functions act as the route planner. You call this function exactly once right before you want the robot to start moving. It takes the distance you want to travel in millimetres (or degrees) and converts it into encoder "ticks," which are the steps measured by the robot's wheel sensors. It adds these ticks to the robot's current position to calculate the exact stopping point. This function does not make the robot move.

The `Kobuki_Drive_Distance` and `Kobuki_Rotate_Distance` functions are the actual driver. Once your destination is set, you must call this function repeatedly in a fast continuous loop. Every time it runs, it calculates the "error," which is simply the distance remaining between where the robot is right now and the finish line. If that remaining distance is close enough to zero, it stops the motors and updates a flag to tell your program the movement is complete.

If the robot hasn't arrived yet, this driver function calculates the motor speed based on how far away the target is. It uses proportional control, meaning the robot will drive quickly when it is far from the goal, and slow down smoothly as the remaining distance shrinks. It also adds a slight minimum speed to this calculation. This extra little push ensures the robot has enough power to overcome the physical friction of the wheels and does not get stuck just a few millimetres short of the finish line.

Behind the scenes, these functions also handle the quirks of the robot's sensors. The wheel encoders can only count up to a certain number before they roll back over to zero, much like an old car's odometer. Because the code uses 16-bit maths, the microcontroller automatically understands this circular counting and will always find the shortest path to the target, even if the sensor numbers wrap around zero while the robot is driving. For example, test your robot by commanding it to rotate for -90 degrees versus 270 degrees and observe how the robot rotate.

To ensure accuracy and stability in the rotations and translations, a simple proportional controller is used for both,shown with the block diagram below. A setpoint is made for the rotation or translation and the speed increasesproportionally depending on the error (or difference) from the current distance/angle. This will be able to handle the Kobuki overshooting a setpoint by changing direction, but more importantly,

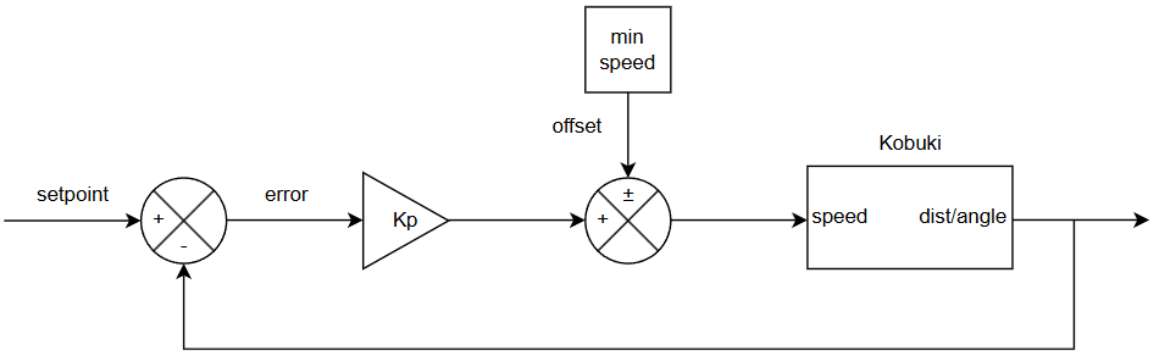


Figure 2: Block diagram of a P controller

There is free choice of the proportional gain (Kp) depending upon the speed profile you wish for your Kobuki inapproaching the setpoint (This has been set in the variable `GAIN_DRIVE` int the code). Shown in the figure below is an illustration of the slope of the line, Kp, and the effect of saturation on the speed:

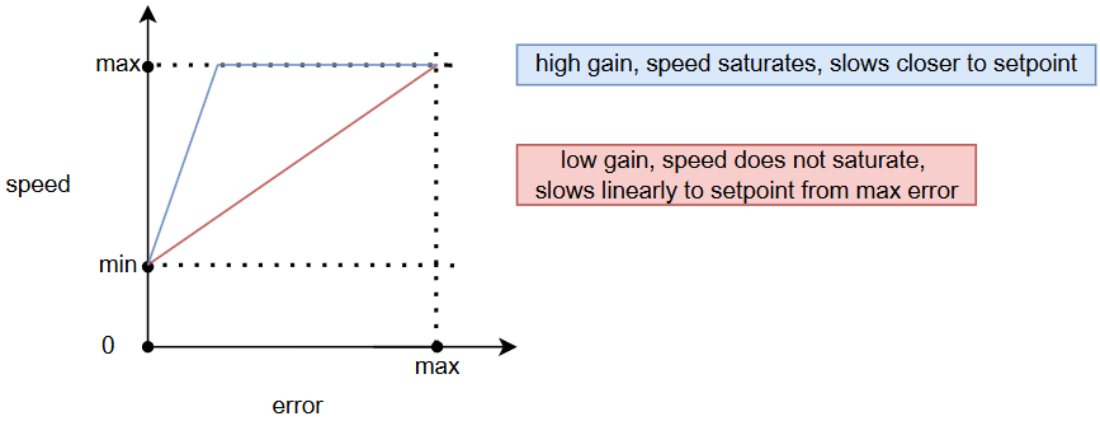


Figure 3: Example of diffrent gains on the speed controller

Follow these steps to update the Kobuki drivers, you can create a new project or update your old one. These wrappers have been completed for you and are particularly invaluable for your project:

1. Increasing buffer feedback size, added in `kobuki.h`. Also set min speed:

```
// Define buffer for feedback, increase from 14 to 26
// for distance and angle feedback from Kobuki
#define SIZE_FEEDBACK 26

// Buffer defintions for payload
#define BUMPER          4
#define WHEEL_DROP      5
#define CLIFF           6
```

```
#define DIST_LSB         7 // only needs 1 wheel
#define DIST_MSB         8
#define BUTTON           13
#define ANGLE_LSB        24 // only needs 1 wheel
#define ANGLE_MSB        25

// Truncated speed
#define SPEED_LIMIT (int16_t)150 //mm/s
#define SEARCH_SPEED ((int16_t)75)  //mm/s
#define STOP         ((int16_t)0)  //mm/s
#define MIN_SPEED    ((int16_t)50)  //mm/s
```

2. Add distance and angle information and flags in Kobuki structure and define Kobuki LEDs enumeration.

```
typedef struct {
    uint8_t     cliff;
    uint8_t     bumper;
    uint8_t     wheeldrop;
    uint8_t     button;
    uint16_t    distance;
    int16_t     angle;
    uint8_t     rotation_complete;
    uint8_t     distance_complete;

} Kobuki_Typedef;

typedef enum uint8_t {OFF, RED, GREEN, YELLOW} Kobuki_LED_Enum
   ;
```

3. Define headers for the following new wrappers:

```
void Kobuki_Rotate_Angle_Setpoint(int16_t current_angle_tick,
   int16_t rot_angle_deg);

void Kobuki_Rotate_Angle(int16_t current_angle_tick);

void Kobuki_Drive_Distance_Setpoint(uint16_t
   current_distance_tick, int16_t distance);

void Kobuki_Drive_Distance(uint16_t current_distance_tick);

// kobuki update LEDs
void Kobuki_Update_LEDs(Kobuki_LED_Enum led_1_colour,
   Kobuki_LED_Enum led_2_colour);
```

4. Declare some parameters in distance and rotation angle calculation in `kobuki.c` and update `Kobuki_Read()` to receive angle and distance feedback:

```
#include <stdlib.h> // include this at the top
```

```
//Parameters for distance and angle calculation
//These can be placed in kobuki.h:
#define PI                          (3.14159)
#define WHEEL_RADIUS                (35.0)      //mm
#define REVOLUTIONS_PER_TICK        (132.0/6545.0)/52.0
#define MM_PER_TICK                 (2.0 * PI * WHEEL_RADIUS *
   REVOLUTIONS_PER_TICK)

#define DEG_PER_TICK                (1.0/100.0)

#define GAIN_ROTATE                 (1.0/0.90)
#define GAIN_DRIVE                  (100.0/100.0)

#define sign(x)                     ((int8_t)((x) >= 0 ? 1 : -1))

// static variables and flags, keep these here in kobuki.c
static volatile int32_t     setpoint_distance_tick;
static volatile int16_t     setpoint_angle_deg;
static          uint8_t     rotation_complete = 0;
static          uint8_t     distance_complete = 0;

// Updated Kobuki_Read() remember to replace the old wrapper
   with this one
void Kobuki_Read (Kobuki_Typedef*kobuki) {
uint16_t    distance_LSB;
uint16_t    distance_MSB;
int16_t     angle_LSB;
int16_t     angle_MSB;


// Init buffer & checksum
uint8_t kobuki_rx_buffer [SIZE_FEEDBACK];
uint8_t checksum_result ;
// Read kobuki Tx data stream
checksum_result = Kobuki_Rx(kobuki_rx_buffer , SIZE_FEEDBACK )
    ;


// If checksum 'checks out', update sensor variables from
   buffer
if ( checksum_result == 0) {
distance_LSB = kobuki_rx_buffer[DIST_LSB];
angle_LSB    = kobuki_rx_buffer[ANGLE_LSB];
distance_MSB = (uint16_t)((uint16_t)kobuki_rx_buffer[DIST_MSB]
    << 8);
angle_MSB    = (int16_t)((int16_t)kobuki_rx_buffer[ANGLE_MSB]
   << 8);


kobuki->bumper = kobuki_rx_buffer[BUMPER];
kobuki->wheeldrop = kobuki_rx_buffer[WHEEL_DROP];
```

```
kobuki->cliff = kobuki_rx_buffer[CLIFF];
kobuki->button = kobuki_rx_buffer [BUTTON];


kobuki -> distance  = distance_LSB | distance_MSB;
kobuki -> angle     = angle_LSB | angle_MSB;
}
// otherwise do not update
else {}

// always update complete flags
kobuki -> rotation_complete =  rotation_complete;
kobuki -> distance_complete =  distance_complete;
}
```

5. Now we add these new wrappers in `kobuki.c`. We can command the Kobuki to rotate at a specific angle. An example call would be `Kobuki_Rotate_Angle(kobuki.angle)`, that is going to take the current gyroscope value,compare to a setpoint value, then set a variable speed dependent on the difference. Unlike the translation, our rotation is a cyclic movement, thus we have multiple solutions. A rotation of 90 degrees is equal to a -270 degree rotation, that is equal to a 450 degree rotation. We aim to perform an 'optimal rotation', that is, forcing either clockwise or counterclockwise to rotate the minimum angle to reach the setpoint. `Kobuki_Rotate_Angle_Setpoint(kobuki.angle,object.angle)` takes a reference angle, then offset this to find and store a setpoint angle.:

```
void Kobuki_Rotate_Angle_Setpoint(int16_t current_angle_tick,
   int16_t rot_angle_deg) {

    volatile int32_t init_angle;

    rotation_complete = 0;

    // initial angle
    init_angle = (int32_t)((double)(current_angle_tick + (
   int32_t)18000) * DEG_PER_TICK);   // [-18000,18000] ->
   [0-360]deg

    // setpoint angle
    setpoint_angle_deg  = (init_angle + rot_angle_deg + 360) %
    360;     // [0-360]

}

void Kobuki_Rotate_Angle(int16_t current_angle_tick) {

    volatile int16_t  speed;
    volatile int16_t  error;
    volatile int32_t    current_angle_deg;

    current_angle_deg = (int32_t)((double)(current_angle_tick
   +(int32_t)18000)*DEG_PER_TICK);
```

```
    error = (int16_t)(setpoint_angle_deg - current_angle_deg);

    // Check for non-optimal rotation angle
    if(abs(error) > 180) {

        // Always rotate with the minimum angle e.g. +270 ->
   -90.
        // Introduces singularity - loses sign and truncates
   error.
        error = -sign(error)*(abs(error) %  180);

        // Special cases in circular buffer. Offset to correct
    error.
        if( (error < 0) && (current_angle_deg <
   setpoint_angle_deg) ) {
            error = -(error + 180);
        }
        else if( (error > 0) && (current_angle_deg >
   setpoint_angle_deg) ) {
            error = -(error - 180);
        // Does this make sense? Is there a simpler way?

        }
    }

    if ( abs(error) <= 0 ){
        rotation_complete = 1;
    }
    else {
        speed = (sign(error) * MIN_SPEED) + (int16_t)(
   GAIN_ROTATE * error);

        Kobuki_Rotate(speed);

    }
}
```

6. Continue placing two new driving wrappers for distance setpoint `kobuki.c`. We can command the Kobuki to drive at a specific distance. `Kobuki_Drive_Distance(kobuki.distance)` takes the current encoder value, compares to a setpoint encoder value, then sets a variable speed dependent on the difference. A special case is given for encoder rollover. Given the sign of the distance from Kobuki Drive Distance Setpoint, this special case could be significantly simplified; however, this is of instructive interest and useful for writing your own functions with alternate functionality. `Kobuki_Drive_Distance_Setpoint(kobuki.distance,(int16 t)object.range)` takes a reference position, then offset this to find and store a setpoint distance:

```
void Kobuki_Drive_Distance_Setpoint(uint16_t
   current_distance_tick, int16_t distance){
```

```
    distance_complete = 0;

    setpoint_distance_tick = current_distance_tick + (int32_t)
   (distance/MM_PER_TICK);

}

void Kobuki_Drive_Distance(uint16_t current_distance_tick) {

    volatile int16_t  speed;
    volatile int16_t  error;
    volatile uint16_t setpoint_alias;

    //standard conditoon
    error = (int16_t)((double)(setpoint_distance_tick -
   current_distance_tick) * MM_PER_TICK);

    setpoint_alias = (uint16_t)( (setpoint_distance_tick + 0
   x10000) % 0x10000);
    if ( ( (setpoint_distance_tick > 0xFFFF) && (
   current_distance_tick < setpoint_alias) ) ||
             ( (setpoint_distance_tick < 0)          && (
   current_distance_tick > setpoint_alias) ) )
    {
        //alternate condition
        error = (int16_t)((double)(setpoint_alias-
   current_distance_tick)* MM_PER_TICK);
    }

    if ( abs(error) <= 2 ){
        distance_complete = 1;
    }
    else {
        speed = (sign(error) * MIN_SPEED) + (int16_t)(
   GAIN_DRIVE * error);

        Kobuki_Drive(speed);

    }

}
```

7. Finally, add Kobuki LEDs update wrapper. We now can control the LEDs on the robot. Use this as you see fit to debug your FSM function later.

```
// This wrapper is based on the General Purpose Output
   information in the Kobuki
// bytestream document
#define EXTERN_PWR      0x30
void Kobuki_Update_LEDs(Kobuki_LED_Enum led_1_colour,
   Kobuki_LED_Enum led_2_colour){
```

```
    //Split LSB & MSB for payload
    uint8_t GPO_lsb = EXTERN_PWR; //keep 5V AND 3.3V on
    uint8_t GPO_msb = led_1_colour | (uint8_t)(led_2_colour <<
    2);

    uint8_t payload[] = {0x0C,0x02,GPO_lsb,GPO_msb};

    Kobuki_Tx(payload,sizeof(payload));

}
```

8. Change your main.c with this test code and check for the driven distance and rotated angle. Note that for this quick FSM test, buttons B0, B1 and B2 command the robot to drive/rotate for certain distances/angles. You will see that the robot will drive/rotate at the set speed, then slow down about half way through due to the effect of P controller. Sanity check other distance/angle as well:

```
#include <stdint.h>

#define STM32G474xx
#include "stm32g4xx.h"
#include "adc.h"
#include "kobuki.h"
#include "uart.h"
#include "ultrasonic.h"

/* Kobuki button values */
#define BUTTON_0_START          0x01U
#define BUTTON_1_STOP           0x02U
#define BUTTON_2_ROTATE           0x04U

// Logic
#define NULL  0
#define TRUE    1

// States
static enum {IDLE, SEARCH, OBSTACLE, AVOID, OBJECT} state;

int main(void) {

    // Both of these only for testing purposes. Not required
    volatile uint16_t ultrasonic_read;
    volatile uint16_t adc_read;

    Kobuki_Typedef kobuki;

    US_Typedef object;

    Kobuki_LED_Enum led_1, led_2;
```

```
    UART_Init();

    ADC_Init();

    Ultrasonic_Init();

  state = IDLE;
    led_1 = OFF;
    led_2 = OFF;

    while (1){

        Kobuki_Read(&kobuki);

        Ultrasonic_Detect(&object);

        // Both of these only for testing purposes. Not
   required
        ultrasonic_read =  Ultrasonic_Read();
        adc_read = ADC_Read(ADC_CHANNEL_1);

        Kobuki_Update_LEDs(led_1, led_2);


        // A TEST FSM
        switch(state){

            case IDLE:

            if ( kobuki.button == BUTTON_0_START ){
                state = SEARCH;
                led_1 = RED;
                Kobuki_Rotate_Angle_Setpoint(kobuki.angle,
   450);
                //Kobuki_Drive_Distance_Setpoint(kobuki.
   distance,100);
            }
            else if ( kobuki.button == BUTTON_1_STOP ){
                state = SEARCH;
                led_1 = RED;
                Kobuki_Rotate_Angle_Setpoint(kobuki.angle,
   270);
                //Kobuki_Drive_Distance_Setpoint(kobuki.
   distance,-100);
            }
            else if ( kobuki.button == BUTTON_2_ROTATE ){
                state = SEARCH;
                led_1 = RED;
                Kobuki_Rotate_Angle_Setpoint(kobuki.angle, 90)
   ;
                //Kobuki_Drive_Distance_Setpoint(kobuki.
```

```
    distance,1000);
                }
                else{
                    Kobuki_Drive(STOP);
                }
                break;

                case SEARCH:
                if ( (kobuki.wheeldrop != NULL) |
                        ((kobuki.bumper + kobuki.cliff) != NULL)
    |
                        (kobuki.rotation_complete == TRUE) |
                        (kobuki.distance_complete == TRUE)
              ){
                    state = IDLE;
                    led_1 = YELLOW;
                }
                else{
                    Kobuki_Rotate_Angle(kobuki.angle);
                    //Kobuki_Drive_Distance(kobuki.distance);
                }
                break;

                case OBSTACLE:
                break;
                case AVOID:
                break;
                case OBJECT:
                break;

            }
        }
}
```

3. Overall Project FSM

The project could be intimidating and you might not know where to start. I have provided an overall FSM below for you to think about. Beware that for each FSM state, you might need to write sub-FSM to achieve certain functions. You don't need to follow this FSM and feel free to modify and improvise as you see fit.

The overall test area specification is shown in Figure 4, but please refer to your project document for full specifications. Our objective is to program the robot to pick the available path and navigate from the starting point to one of the deposit locations.

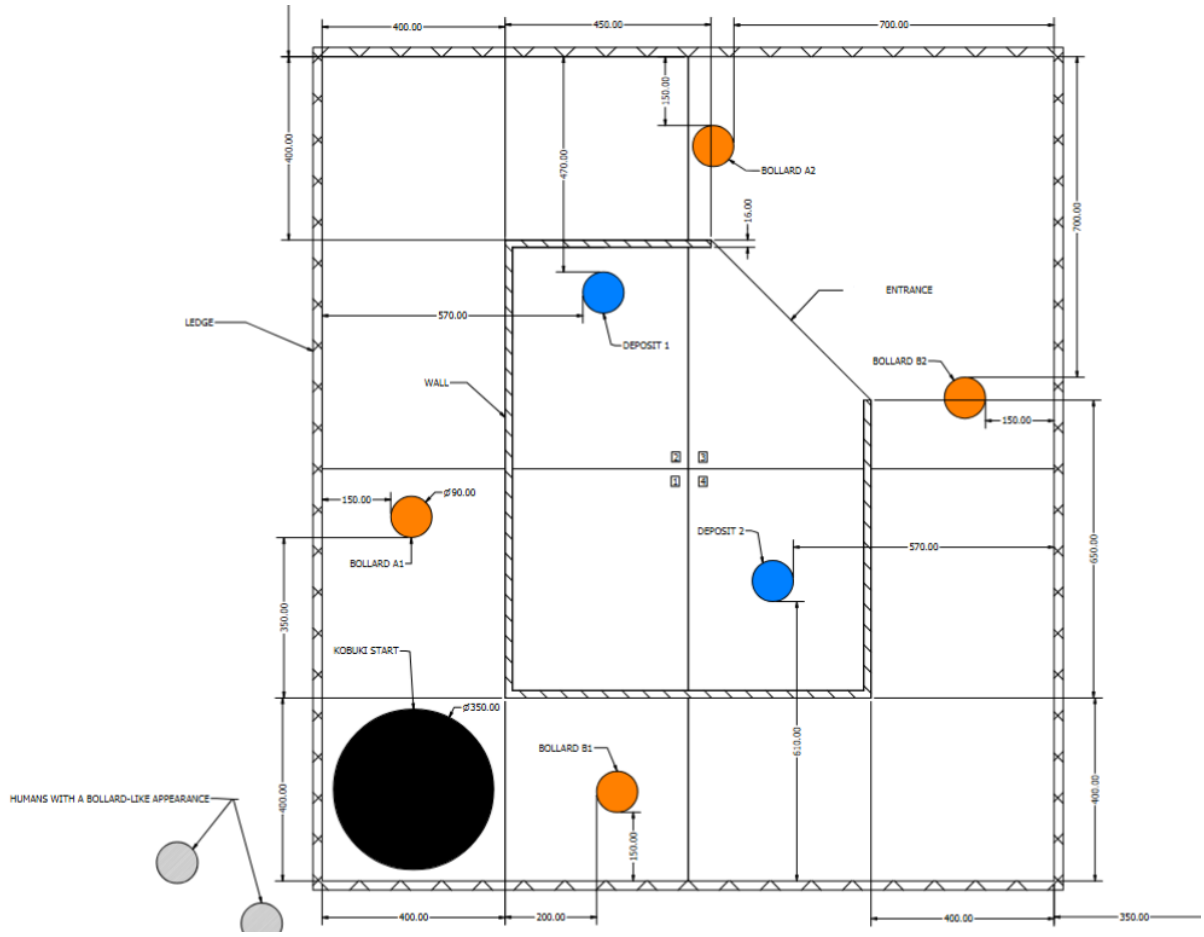


Figure 4: Area Schematic

There are many different steps in your program you need to think about in order to achieve this. You can reference the following steps, with reference to Figure 5:

1. Your robot will be at a random orientation, align with the blocking bollard and identify the correct path to go. Corresponding to SEARCH and ORIENT FSM states.
2. Navigate the path until you reach section 3 (denoted by the number on the platform). Corresponding to PATHFIND FSM states.
3. Orientate into the storage unit and drive into the fenced area. Corresponding to STORAGE FSM states.
4. Search for the deposit bollard and drive until close enough. Corresponding to DEPOSIT FSM states.

Obviously you can break these down into more steps as you see fit, but this should give you a general idea on where to start.

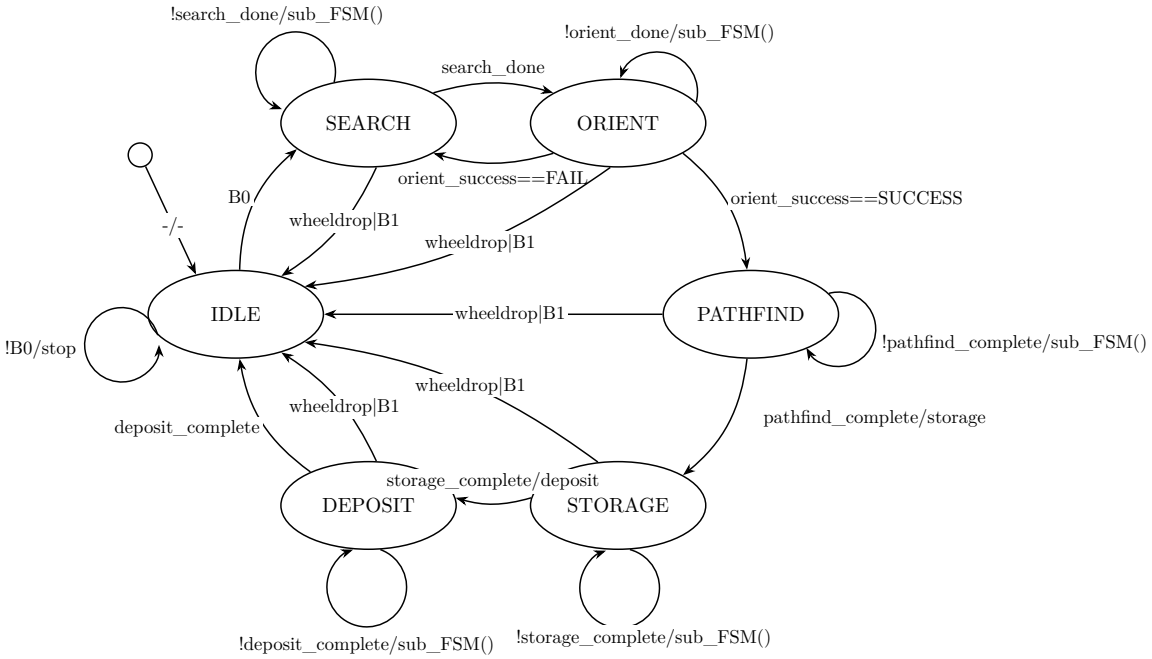


Figure 5: *Main Finite State Machine for basic algorithm flow of AMR operation.*